

Inheritance can create classes with complexity and contradiction. It's often favorable to use composition instead.

The `isinstance()` and `issubclass()` Functions

When we need to know the type of an object, we can pass the object to the built-in `type()` function, as described in the previous chapter. But if we're doing a type check of an object, it's a better idea to use the more flexible `isinstance()` built-in function. The `isinstance()` function will return `True` if the object is of the given class *or a subclass of the given class*. Enter the following into the interactive shell:

```
>>> class ParentClass:
...     pass
...
>>> class ChildClass(ParentClass):
...     pass
...
>>> parent = ParentClass() # Create a ParentClass object.
>>> child = ChildClass() # Create a ChildClass object.
>>> isinstance(parent, ParentClass)
True
>>> isinstance(parent, ChildClass)
False
❶ >>> isinstance(child, ChildClass)
True
❷ >>> isinstance(child, ParentClass)
True
```

Notice that `isinstance()` indicates that the `ChildClass` object in `child` is an instance of `ChildClass` ❶ and an instance of `ParentClass` ❷. This makes sense, because a `ChildClass` object “is a” kind of `ParentClass` object.

You can also pass a tuple of class objects as the second argument to see whether the first argument is one of any of the classes in the tuple:

```
>>> isinstance(42, (int, str, bool)) # True if 42 is an int, str, or bool.
True
```
